

Arsitektur Final — Platform Bantuan Mahasiswa + Admin Dropship/Affiliate

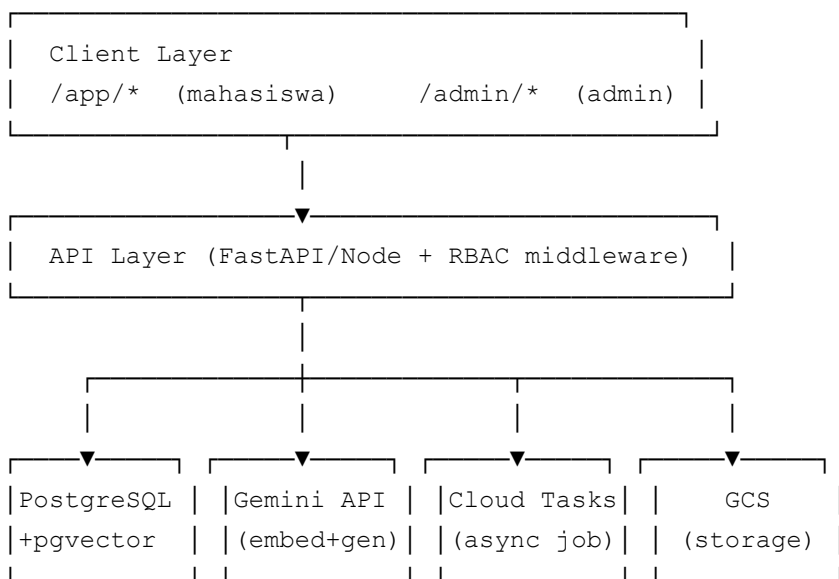
1. Context & Goals

Satu platform, dua sisi:

- **Public (mahasiswa):** Web app buat cek plagiarisme, parafrase, benerin format & penomoran laporan, typo/grammar — dengan report perubahan biar user bisa review sebelum final.
- **Admin-only (hidden, RBAC):** Dashboard buat ngontrol bisnis dropship (produk sendiri) + tracking affiliate (kamu jadi affiliate di platform lain kayak Shopee Affiliate/TikTok Shop Affiliate).

Dua sisi ini berbagi infrastruktur (auth, database, AI engine, deployment) tapi terpisah total secara akses — mahasiswa tidak pernah tahu modul admin itu ada.

2. High-Level Architecture



Deploy: Cloud Run, semua di GCP — sejalan sama stack T-WASH.

3. Module A — Student Tool (Public)

Fitur: Cek Plagiarisme · Parafrase Otomatis · Cek & Benerin Format Laporan · Benerin Typo & Grammar · Benerin Penomoran · Report Perubahan

Flow: Upload dokumen → pilih mode (Cepat/Advanced) → processing → review report → accept/reject per item → download hasil final

Fitur	Pendekatan Teknis
Plagiarisme	Embedding self-hosted (BGE-M3/EmbeddingGemma-300M, bukan Gemini API) + pgvector, bandingin ke corpus internal (mode cepat) + API eksternal (Copyleaks dkk) buat nyari ke web luar (mode advanced)
Parafrase	Gemini API, prompt di-tune biar istilah teknis & kutipan langsung nggak ikut keparafrase
Format & Penomoran	Parsing docx (python-docx/mammoth) → validasi vs rule template kampus (config JSON) → auto-fix, resequence heading/gambar/tabel/referensi + update cross-reference
Typo & Grammar	LanguageTool (B.Indo) / Gemini
Report	Diff before/after, dikategorikan per jenis perubahan. Format/numbering auto-fix langsung; plagiarism/parafrase wajib direview manual dulu (butuh human judgment)

Data model: Documents → Versions → Chunks (embedding) → PlagiarismMatches, ParaphraseEdits, FormatIssues, NumberingFixes → ChangeReports

Diferensiator: report terintegrasi (vs Turnitin+Quillbot+Grammarly yang terpisah-pisah), self-plagiarism check antar-bab punya user sendiri, version history progress revisi.

4. Module B — Admin Panel (Hidden, role: admin only)

B1. Dropship — Manajemen Pesanan & Stok (produk sendiri)

- Pipeline pesanan: Baru → Diteruskan ke supplier → Dikirim → Selesai/Retur
- Auto-routing ke supplier terbaik (harga/stok/kecepatan) kalau satu produk ada beberapa supplier
- Stok: update manual dulu, API supplier nyusul kalau ada yang support

- Alert stok menipis + reorder point
- Supplier scoring (reliability/speed) → auto-prefer di routing
- Demand forecasting (AI prediksi produk yang bakal laris duluan)

B2. Affiliate — Link, Tracking & Komisi (kamu sebagai affiliate di platform lain)

- Link management per platform (Shopee Affiliate, TikTok Shop Affiliate, network kayak Involve Asia/Accesstrade) & per produk/campaign
- Redirect/cloaking layer sendiri buat click tracking (platform nggak expose API individual buat affiliate)
- Commission reconciliation via import CSV/Excel (laporan berkala dari platform — nggak ada API resmi buat ini)
- Analytics performa per platform/produk/konten

B3. Shared — Auto-generate Konten (AI)

- Reuse Gemini engine yang sama dari Module A → judul/deskripsi produk (dropship) + caption promosi (affiliate), gaya beda per channel
- Batch generate buat seluruh katalog sekaligus
- Template brand voice biar hasilnya konsisten

B4. Dashboard Analitik

- Revenue/profit dipisah 2 stream: dropship (per produk/supplier) vs affiliate (per platform/link)
- Top performer (produk, supplier, platform, konten)
- Trend over time + export laporan mingguan/bulanan (auto-kirim ke email admin)
- Smart recommendation: platform/produk mana yang paling worth digenjot

Data model: Suppliers , Products , Orders (dropship) · AffiliatePlatforms , AffiliateLinks , Clicks , CommissionRecords (affiliate) · ContentJobs (shared B1+B2)

5. Shared Infrastructure

Layer	Pilihan	Alasan

Backend	FastAPI / Node.js	Cepat buat MVP, developer solo
Database	PostgreSQL + pgvector	Satu DB buat relational + vector search, nggak perlu vector DB terpisah
AI	Gemini API	Udah familiar dari T-WASH (RAG), dipakai buat embedding + generation di kedua modul
Storage	GCS	Upload dokumen mahasiswa
Queue	Cloud Tasks	Proses async: mode advanced (plagiarism-check) & batch content generation
Deploy	Cloud Run	Sejalan T-WASH, serverless, scale-to-zero pas sepi traffic
Auth	RBAC (role: mahasiswa/admin)	Satu sistem auth, akses admin dipisah total di level middleware + route namespace

6. Consolidated Database Schema

- **Shared:** `Users` (field: `role` → `mahasiswa/admin`)
- **Module A:** `Documents` → `Versions` → `Chunks` → `PlagiarismMatches`, `ParaphraseEdits`, `FormatIssues`, `NumberingFixes` → `ChangeReports`
- **Module B1:** `Suppliers` ← `Products` → `Orders`
- **Module B2:** `AffiliatePlatforms` → `AffiliateLinks` → `Clicks` → `CommissionRecords`
- **Shared B1+B2:** `ContentJobs` (dipakai `Products` & `AffiliateLinks`)

7. Security & Separation

- Route namespace terpisah: `/app/*` (mahasiswa) vs `/admin/*` (kamu doang)
- RBAC middleware cek role di setiap request ke `/admin/*` — block di level API, bukan cuma disembunyikan di UI
- Anomaly detection di `Clicks` (cegah self-referral/abuse ke tracking affiliate-mu sendiri)
- Subdomain terpisah opsional (`admin.domainkamu.com`) sebagai lapisan tambahan
- Satu database, tapi table prefix beda (`app_*` vs `biz_*`) biar gampang dipisah jadi service sendiri kalau nanti perlu di-scale terpisah

8. Gap Analysis — Biar Beneran Advance & Layak

8.1 Module B (Admin) — Kontrol & Operasional

Jujur, arsitektur di atas **solid buat MVP**, tapi belum “paling advance” — dan “timeless” itu klaim yang susah dipegang buat arsitektur manapun (teknologi pasti butuh direvisit seiring waktu). Ini yang masih kurang:

Biar kontrol admin beneran gampang (via web):

- Notification layer: WhatsApp Business API / Telegram bot → alert real-time (order baru, stok abis, komisi cair) tanpa harus buka dashboard terus
- Real-time feed: WebSocket/SSE buat order & analytics, bukan polling/refresh manual
- KPI dashboard di homepage admin — angka penting langsung kelihatan, bukan cuma tabel mentah
- Bulk/quick actions: select multiple order → approve/route sekaligus, tanpa masuk detail satu-satu
- Global search + filter di semua modul
- Audit log: siapa ubah apa kapan (penting kalau nanti nggak kerja sendirian lagi)

Biar “advance” secara sistem:

- Caching layer (Redis) — dashboard/analytics yang sering diakses jangan hit DB tiap kali
- Automation rules (if-this-then-that): stok < X → auto-notify supplier; komisi > threshold → auto-request payout
- Observability: logging terstruktur + error tracking (Sentry) — biar tau ada yang error tanpa nunggu sadar manual

Soal klaim “timeless”:

- Stack sekarang lumayan terikat ke GCP + Gemini API (Cloud Run, Cloud Tasks, GCS) — cepat buat MVP, tapi effort besar kalau nanti pindah cloud/provider AI
- Yang beneran timeless itu prinsipnya (separation of concerns, RBAC, modularitas per domain) — itu udah diterapkan. Pilihan teknologi spesifiknya tetap akan butuh direvisit seiring platform tumbuh

Prioritas kalau mau nambah: notification bot dulu (paling kerasa buat kontrol harian) → baru caching & observability pas traffic mulai ada.

8.2 Module A (Student Tool) — Fitur & Integritas Akademik

Sama kayak Module B, ini juga belum “layak & advance” sepenuhnya. Gap yang paling kerasa:

Fitur yang kelewat/belum ada:

- **Cek & benerin Daftar Pustaka** (APA/IEEE/Vancouver, sesuai gaya sitasi kampus) — salah satu pain point terbesar nulis skripsi, tapi kemarin nggak masuk sama sekali
- **Auto-sync Daftar Isi** dengan heading & nomor halaman aktual (sering kacau, mahasiswa update manual)
- **Template learning** — sekarang rule format itu hardcode per kampus (JSON config manual). Biar scalable ke kampus lain, harusnya bisa upload 1 contoh dokumen yang udah sesuai → sistem extract rule-nya sendiri
- **Domain dictionary** — istilah teknis/serapan asing bakal salah keflag jadi typo kalau cuma pakai generic checker
- **Multi-format** — sekarang cuma disebut docx, padahal submission akhir sering PDF, atau mahasiswa nulis di Google Docs/LaTeX

Satu isu desain yang lebih penting — interaksi plagiarism-check x parafrase:

Kalau alurnya “similarity tinggi → auto-paraphrase → skor turun → submit”, ada dua masalah:

1. Nggak nyelesain akar masalahnya — mahasiswa tetep nggak nulis pake pemahaman sendiri, cuma nyamarin angka
2. Secara teknis makin nggak reliable — plagiarism checker resmi kampus (Turnitin dkk) sekarang udah punya AI-paraphrase detection, jadi “diparafrase doang” belum tentu lolos beneran

Desain yang lebih kuat: pas ada bagian keflag similarity tinggi, tool arahin ke sitasi yang bener (auto-suggest kutipan+sumber sesuai gaya sitasi kampus), bukan cuma tombol “parafrase sampe angkanya turun”. Parafrase tetap berguna, tapi buat rewriting dengan pemahaman — bukan buat evasion.

Report/UX:

- Belum ada severity ranking — plagiarism 40% harusnya jauh lebih menonjol dibanding typo koma, sekarang semua kelihatan sama rata
- Belum ada overall readiness score (misal “82% siap submit”)

9. Cost Control — Gemini API & Budget Safety

Kritis: platform gratis + AI API pay-per-token + no cap = resep buat “meledak” kayak kasus Cursor — bedanya sekarang kamu di posisi platform-nya, bukan customer-nya.

Kenapa bisa meledak:

- 1 skripsi = 50-150 halaman → di-chunk buat embedding + potensial full-document paraphrase = puluhan-ratusan ribu token per dokumen, per user
- Gratis + no quota = nggak ada rem alami; sekali platform rame, cost naik linear tanpa revenue penyeimbang
- Per April 2026, model Gemini Pro-tier udah nggak ada lagi di free tier — full paid dari token pertama. Cuma Flash & Flash-Lite yang masih ada jatah gratis (rate limit ketat, reset harian)
- Kalau semua request dilempar ke model Pro (bukan cuma yang bener-bener butuh), itu bisa ~20-24x lebih mahal dibanding Flash-Lite buat kerjaan yang sama

Fix konkret:

Strategi	Implementasi	Efek
Model routing	Flash-Lite/Flash jadi default (typo, grammar, klasifikasi similarity) — Pro cuma buat kasus low-confidence/butuh reasoning dalam	Selisih sampai ~20x di output cost
Batch API	Proses mode Advanced (yang emang udah async by design) lewat Batch API, bukan real-time	~50% lebih murah
Context caching	Cache system prompt/instruksi yang dipakai berulang (rule template, instruksi parafrase)	Cached input ~10x lebih murah
Self-hosted embedding	Ganti Gemini embedding API dengan model open-source (BGE-M3 atau EmbeddingGemma-300M) yang di-host sendiri	Hilangin cost embedding API sepenuhnya — trade-off di operational complexity (urus GPU/inference sendiri)
Kuota per user	Cap dokumen/bulan atau halaman/dokumen buat tier gratis, verifikasi email kampus (1 akun = 1 mahasiswa asli)	Nutup celah abuse/bot

Budget hard-cap	Set budget alert + auto-disable/degrade di GCP billing — bukan cuma alert, circuit breaker beneran	Cegah kejadian kayak Cursor, versi kamu jadi operator-nya
-----------------	--	---

Build vs buy — model AI:

- Model generatif (parafrase/grammar) → **jangan** dibikin/train sendiri. Training data pasangan (before/after) buat B.Indo akademik belum ada, kualitas kemungkinan kalah dari Gemini, dan hosting GPU 24/7 itu fixed cost yang bisa lebih mahal dari API kalau traffic belum stabil
- Embedding (buat plagiarism-check) → **bisa & disarankan** self-hosted. Nggak butuh training data, model open-source (BGE-M3/EmbeddingGemma-300M) kualitasnya udah sebanding API komersial buat multilingual
- Fine-tuning kecil buat task spesifik → dipertimbangkan belakangan, setelah ada data usage real — bukan diputusin di awal tanpa data

Perhatian tambahan: konten yang lewat free tier bisa dipakai Google buat improve produk mereka — worth dipikirin karena ini dokumen skripsi mahasiswa yang notabene belum publish. Free tier juga umumnya diperuntukkan buat prototyping, bukan serve traffic user asli di production — begitu platform live, kemungkinan besar tetap butuh paid tier, cuma dengan model routing yang bener biar tetap murah.

Keputusan final — Freemium: 100% gratis = kamu nombokin ~Rp 2 juta/bulan selamanya (biaya tetap), karena biaya AI variable per dokumen tetap kecil dibanding itu. Freemium jauh lebih masuk akal — format/typo/numbering tetap gratis unlimited (rule-based, ~Rp0), parafrase/plagiarism-advance dikasih kuota gratis + paid tier.

- **Harga:** Rp 40.000 sekali bayar/skripsi, atau Rp 20.000/bulan buat yang mau nyicil (efektif Rp 60.000 kalau rata-rata dipakai 3 bulan — lebih tinggi dari sekali bayar, tapi risiko cancel lebih cepat lebih besar)
- **Limit paid tier:** kuota 300 halaman/bulan (reset tiap bulan), setara ~3-4x skripsi penuh — abis kuota = tunggu reset atau upgrade. Bukan buat ngirit (biaya variable-nya emang udah nyaris nol), tapi nyegah reselling/abuse
- **Free tier:** kuota parafrase 1 halaman/hari, reset tiap hari (simplen diinget, dan biayanya kecil banget: ~Rp 104/bulan per free user kalau dipakai maksimal tiap hari — 1.000 free user aktif penuh cuma ~Rp 104rb/bulan, jauh di bawah biaya tetap)
- **Kualitas parafrase:** paid tier pakai Gemini Flash (bukan Flash-Lite) biar hasilnya lebih bagus
- Margin ~99,6% (pemakaian normal), tetap ~97,1% bahkan kalau kuota 300 halaman

dipakai habis sebulan penuh. Breakeven 50 user/bulan. Detail kalkulasi & skenario di file `margin-model.xlsx`.

10. Yang Masih Perlu Diputusin

- Nama platform & domain
- Rule template format laporan spesifik (1 kampus atau multi-template)
- Platform affiliate spesifik yang dipakai (biar redirect/tracking-nya presisi per platform)
- Modul mana yang dikerjakan duluan buat MVP